



HP Fortify Audit Workbench

---

# Developer Workbook

---

Onboarding.WebAPI

# Table of Contents

[Executive Summary](#)

[Project Description](#)

[Issue Breakdown by Fortify Categories](#)

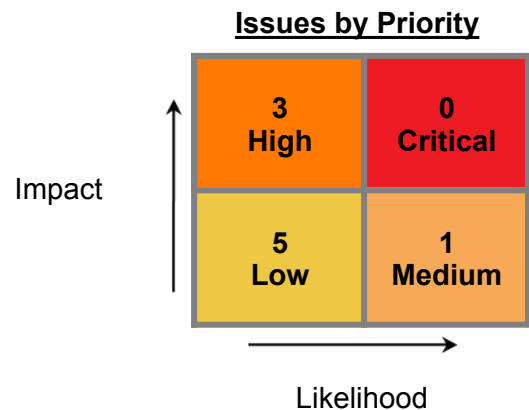
[Results Outline](#)

# Executive Summary

This workbook is intended to provide all necessary details and information for a developer to understand and remediate the different issues discovered during the Onboarding.WebAPI project audit. The information contained in this workbook is targeted at project managers and developers.

This section provides an overview of the issues uncovered during analysis.

|                  |                     |
|------------------|---------------------|
| Project Name:    | Onboarding.WebAPI   |
| Project Version: |                     |
| SCA:             | Results Present     |
| WebInspect:      | Results Not Present |
| SecurityScope:   | Results Not Present |
| Other:           | Results Not Present |



## Top Ten Critical Categories

This project does not contain any critical issues

## Project Description

This section provides an overview of the HP Fortify scan engines used for this project, as well as the project meta-information.

### SCA

|                               |                       |                        |           |
|-------------------------------|-----------------------|------------------------|-----------|
| <b>Date of Last Analysis:</b> | Dec 13, 2016, 3:12 PM | <b>Engine Version:</b> | 6.40.0089 |
| <b>Host Name:</b>             | hendryw-acer          | <b>Certification:</b>  | VALID     |
| <b>Number of Files:</b>       | 263                   | <b>Lines of Code:</b>  | 6,365     |

## Issue Breakdown by Fortify Categories

The following table depicts a summary of all issues grouped vertically by Fortify Category. For each category, the total number of issues is shown by Fortify Priority Order, including information about the number of audited issues.

| Category                                            | Fortify Priority (audited/total) |       |        |       | Total Issues |
|-----------------------------------------------------|----------------------------------|-------|--------|-------|--------------|
|                                                     | Critical                         | High  | Medium | Low   |              |
| ASP.NET Bad Practices: Leftover Debug Code          | 0                                | 0     | 0      | 0 / 1 | 0 / 1        |
| ASP.NET Misconfiguration: Debug Information         | 0                                | 0     | 0 / 1  | 0     | 0 / 1        |
| Dead Code: Unused Field                             | 0                                | 0     | 0      | 0 / 1 | 0 / 1        |
| Dead Code: Unused Method                            | 0                                | 0     | 0      | 0 / 1 | 0 / 1        |
| Password Management: Password in Configuration File | 0                                | 0 / 3 | 0      | 0     | 0 / 3        |
| System Information Leak                             | 0                                | 0     | 0      | 0 / 2 | 0 / 2        |

# Results Outline

## ASP.NET Bad Practices: Leftover Debug Code (1 issue)

### Abstract

Debug code can create unintended entry points in a deployed web application.

### Explanation

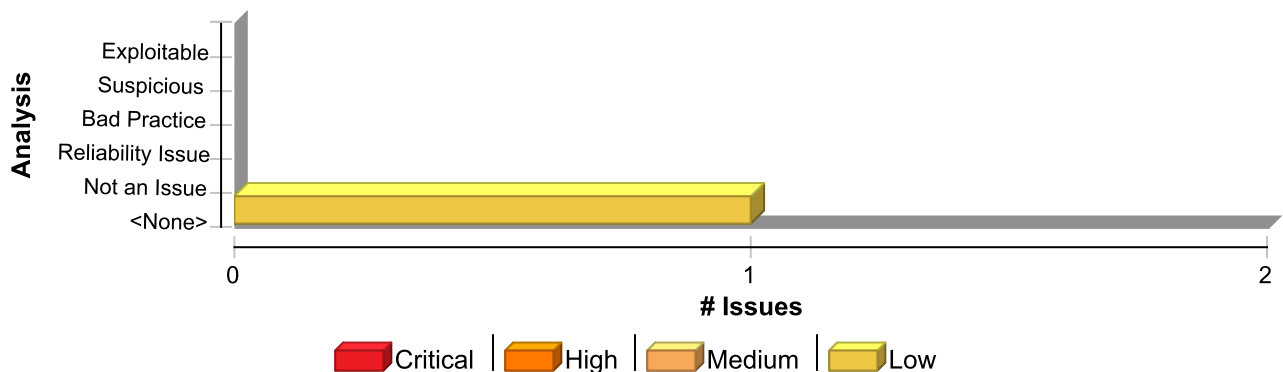
A common development practice is to add "back door" code specifically designed for debugging or testing purposes that is not intended to be shipped or deployed with the application. When this sort of debug code is accidentally left in the application, the application is open to unintended modes of interaction. These back door entry points create security risks because they are not considered during design or testing and fall outside of the expected operating conditions of the application.

The most common example of forgotten debug code is a `Main()` method appearing in a web application. Although this is an acceptable practice during product development, classes that are part of a production ASP.NET application should not define a `Main()`.

### Recommendation

Remove debug code before deploying a production version of an application. Regardless of whether a direct security threat can be articulated, it is unlikely that there is a legitimate reason for such code to remain in the application after the early stages of development.

### Issue Summary



### Engine Breakdown

|                                            | SCA | WebInspect | SecurityScope | Total |
|--------------------------------------------|-----|------------|---------------|-------|
| ASP.NET Bad Practices: Leftover Debug Code | 1   | 0          | 0             | 1     |
| Total                                      | 1   | 0          | 0             | 1     |

|                                                                                                                                                         |            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>ASP.NET Bad Practices: Leftover Debug Code</b>                                                                                                       | <b>Low</b> |
| <b>Package: Onboarding.EmailAlert</b>                                                                                                                   |            |
| <b>D:/Work2/VSTS_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.EmailAlert/Program.cs, line 18 (ASP.NET Bad Practices: Leftover Debug Code)</b> | <b>Low</b> |

#### Issue Details

**Kingdom:** Encapsulation

**Scan Engine:** SCA (Structural)

#### Sink Details

**Sink:** Function: Main

**Enclosing Method:** Main()

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.EmailAlert/Program.cs:18

**Taint Flags:**

No snippet available

# ASP.NET Misconfiguration: Debug Information (1 issue)

## Abstract

Debugging messages help attackers learn about the system and plan a form of attack.

## Explanation

ASP .NET applications can be configured to produce debug binaries. These binaries give detailed debugging messages and should not be used in production environments. The debug attribute of the tag defines whether compiled binaries should include debugging information.

The use of debug binaries causes an application to provide as much information about itself as possible to the user. Debug binaries are meant to be used in a development or testing environment and can pose a security risk if they are deployed to production. Attackers can leverage the additional information they gain from debugging output to mount attacks targeted on the framework, database, or other resources used by the application.

## Recommendation

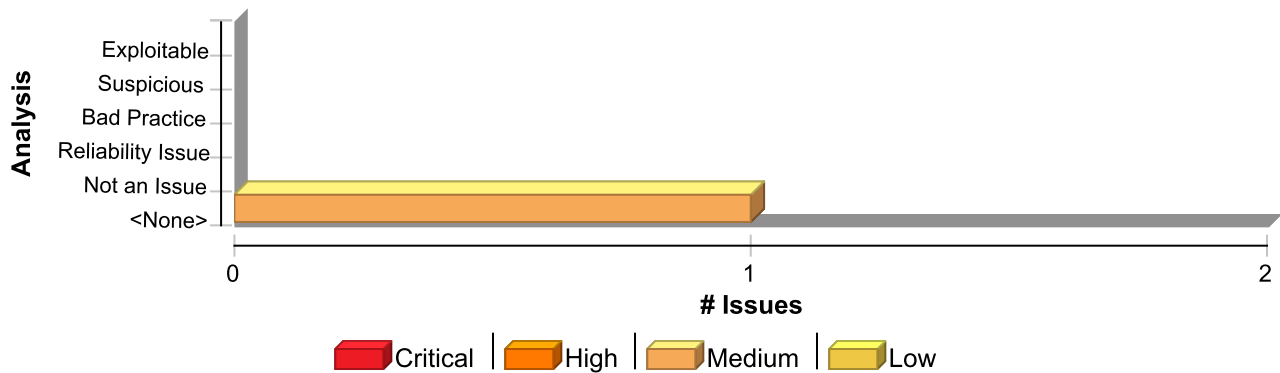
Always compile production binaries without debug enabled. This can be accomplished by setting the debug attribute to false on the tag in your application's configuration file, as follows:

...

...

Setting the debug attribute to false is necessary for creating a secure application. However, it is important that your application does not leak important system information in other ways. Ensure that your code does not unnecessarily expose system information that could be useful to an attacker.

## Issue Summary





## Engine Breakdown

|                                             | SCA      | WebInspect | SecurityScope | Total    |
|---------------------------------------------|----------|------------|---------------|----------|
| ASP.NET Misconfiguration: Debug Information | 1        | 0          | 0             | 1        |
| <b>Total</b>                                | <b>1</b> | <b>0</b>   | <b>0</b>      | <b>1</b> |

### ASP.NET Misconfiguration: Debug Information

Medium

Package: D:.Work2.VSTS\_Workspace.Onboarding.SourceCodes.Onboarding.Onboarding.WebAPI

D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config, line 18 (ASP.NET Misconfiguration: Debug Information)

Medium

### Issue Details

**Kingdom:** Environment

**Scan Engine:** SCA (Configuration)

### Sink Details

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config:18

**Taint Flags:**

No snippet available

## Dead Code: Unused Field (1 issue)

### Abstract

This field is never used directly or indirectly by a public method.

### Explanation

This field is never accessed, except perhaps by dead code. Dead code is defined as code that is never directly or indirectly executed by a public method. It is likely that the field is simply vestigial, but it is also possible that the unused field points out a bug.

**Example 1:** The field named `g lue` is not used in the following class. The author of the class has accidentally put quotes around the field name, transforming it into a string constant.

```
public class Dead {  
    string glue;  
  
    public string GetGlue() {  
        return "glue";  
    }  
}
```

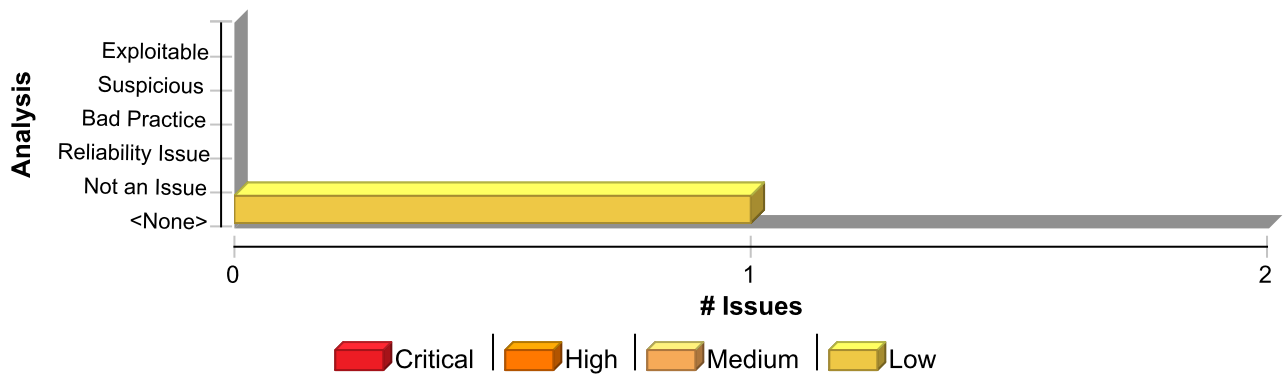
**Example 2:** The field named `g lue` is used in the following class, but only from a method that is never called by a public method.

```
public class Dead {  
    string glue;  
  
    private string GetGlue() {  
        return glue;  
    }  
}
```

### Recommendation

In general, you should repair or remove dead code. To repair dead code, execute the dead code directly or indirectly through a public method. Dead code causes additional complexity and maintenance burden without contributing to the functionality of the program.

### Issue Summary



## Engine Breakdown

|                         | SCA      | WebInspect | SecurityScope | Total    |
|-------------------------|----------|------------|---------------|----------|
| Dead Code: Unused Field | 1        | 0          | 0             | 1        |
| <b>Total</b>            | <b>1</b> | <b>0</b>   | <b>0</b>      | <b>1</b> |

|                                |            |
|--------------------------------|------------|
| <b>Dead Code: Unused Field</b> | <b>Low</b> |
|--------------------------------|------------|

**Package: Onboarding.DataProvider**

|                                                                                                                                                          |            |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>D:/Work2/VSTS_Workspace/Onboarding/SourceCodes/Onboarding/backup/Onboarding.DataProvider_Onboarding4/Common.cs, line 12 (Dead Code: Unused Field)</b> | <b>Low</b> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------|

### Issue Details

**Kingdom:** Code Quality

**Scan Engine:** SCA (Structural)

### Sink Details

**Sink:** Field: instanceExists

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/backup/Onboarding.DataProvider\_Onboarding4/Common.cs:12

**Taint Flags:**

No snippet available

## Dead Code: Unused Method (1 issue)

### Abstract

This method is not reachable from any method outside the class.

### Explanation

This method is never called or is only called from other dead code. Dead code is defined as code that is never directly or indirectly executed by a public method.

**Example 1:** In the following class, the method DoWork( ) can never be called.

```
public class Dead {  
    private void DoWork() {  
        Console.Write("doing work");  
    }  
    public static void Main(string[] args) {  
        Console.Write("running Dead");  
    }  
}
```

**Example 2:** In the following class, two private methods call each other, but since neither one is ever invoked from anywhere else, they are both dead code.

```
public class DoubleDead {  
    private void DoTweedledee() {  
        DoTweedledumb();  
    }  
    private void DoTweedledumb() {  
        DoTweedledee();  
    }  
    public static void Main(string[] args) {  
        Console.Write("running DoubleDead");  
    }  
}
```

(In this case it is a good thing that the methods are dead: invoking either one would cause an infinite loop.)

### Recommendation

A dead method may indicate a bug in dispatch code.

**Example 3:** If method is flagged as dead named GetWitch( ) in a class that also contains the following dispatch method, it may be because of a copy-and-paste error. The 'w' case should return GetWitch( ) not GetMummy( ).

```
public ScaryThing GetScaryThing(char st) {  
    switch(st) {  
        case 'm':  
            return GetMummy();  
    }  
}
```

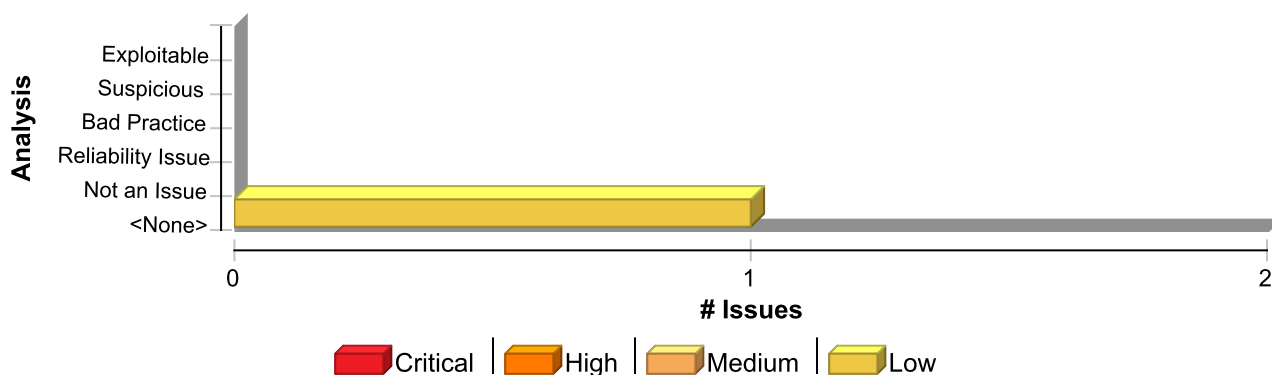
```

    case 'w':
        return GetMummy();
    default:
        return GetBlob();
}
}

```

In general, you should repair or remove dead code. To repair dead code, execute the dead code directly or indirectly through a public method. Dead code causes additional complexity and maintenance burden without contributing to the functionality of the program.

## Issue Summary



## Engine Breakdown

|                          | SCA      | WebInspect | SecurityScope | Total    |
|--------------------------|----------|------------|---------------|----------|
| Dead Code: Unused Method | 1        | 0          | 0             | 1        |
| <b>Total</b>             | <b>1</b> | <b>0</b>   | <b>0</b>      | <b>1</b> |

|                                                                                                                                                                        |            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>Dead Code: Unused Method</b>                                                                                                                                        | <b>Low</b> |
| <b>Package: __ASP</b>                                                                                                                                                  |            |
| <b>C:/Users/hendryw.SMUSTF/AppData/Local/Temp/Temporary ASP.NET Files/onboarding.webapi/bdc96114/7cf1237d/App_global.asax.1.cs, line 30 (Dead Code: Unused Method)</b> | <b>Low</b> |

### Issue Details

**Kingdom:** Code Quality

**Scan Engine:** SCA (Structural)

### Sink Details

**Sink:** Function: Create\_ASP\_global\_asax

**Enclosing Method:** Create\_ASP\_global\_asax()

**File:** C:/Users/hendryw.SMUSTF/AppData/Local/Temp/Temporary ASP.NET Files/onboarding.webapi/bdc96114/7cf1237d/App\_global.asax.1.cs:30

**Taint Flags:**

No snippet available

## Password Management: Password in Configuration File (3 issues)

### Abstract

Storing a plaintext password in a configuration file could result in a system compromise.

### Explanation

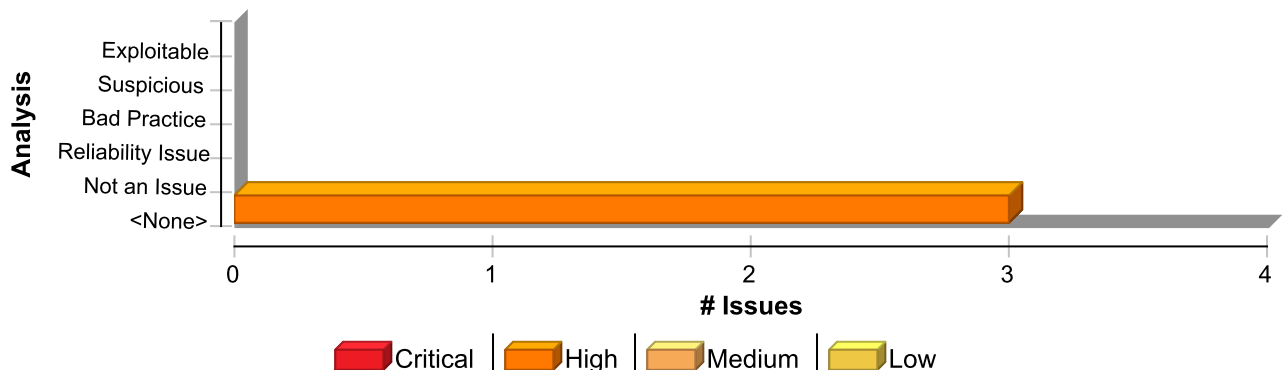
Storing a plaintext password in a configuration file allows anyone who can read the file access to the password-protected resource. Developers sometimes believe that they cannot defend the application from someone who has access to the configuration, but this attitude makes an attacker's job easier. Good password management guidelines require that a password never be stored in plaintext.

### Recommendation

A password should never be stored in plaintext. Instead, the password should be entered by an administrator when the system starts. If that approach is impractical, a less secure but often adequate solution is to obfuscate the password and scatter the de-obfuscation material around the system so that an attacker has to obtain and correctly combine multiple system resources to decipher the password.

Microsoft(R) provides a tool that can be used in conjunction with the Windows Data Protection application programming interface (DPAPI) to protect sensitive application entries in configuration files [1].

### Issue Summary



### Engine Breakdown

|                                                     | SCA      | WebInspect | SecurityScope | Total    |
|-----------------------------------------------------|----------|------------|---------------|----------|
| Password Management: Password in Configuration File | 3        | 0          | 0             | 3        |
| <b>Total</b>                                        | <b>3</b> | <b>0</b>   | <b>0</b>      | <b>3</b> |

### Password Management: Password in Configuration File

High

Package: D:\Work2\VSTS\_Workspace\Onboarding.SourceCodes\Onboarding.Onboarding.WebAPI

D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config, line 89 (Password Management: Password in Configuration File)

High

### Issue Details

Kingdom: Environment

|                                                                                                                                                              |             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>Password Management: Password in Configuration File</b>                                                                                                   | <b>High</b> |
| <b>Package: D:\Work2\VSTS_Workspace\Onboarding\SourceCodes\Onboarding\Onboarding.WebAPI</b>                                                                  |             |
| <b>D:\Work2\VSTS_Workspace\Onboarding\SourceCodes\Onboarding\Onboarding.WebAPI\Web.config, line 89 (Password Management: Password in Configuration File)</b> | <b>High</b> |

**Scan Engine:** SCA (Configuration)

#### Sink Details

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config:89

**Taint Flags:**

No snippet available

|                                                                                                                                                              |             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>D:\Work2\VSTS_Workspace\Onboarding\SourceCodes\Onboarding\Onboarding.WebAPI\Web.config, line 90 (Password Management: Password in Configuration File)</b> | <b>High</b> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|

#### Issue Details

**Kingdom:** Environment

**Scan Engine:** SCA (Configuration)

#### Sink Details

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config:90

**Taint Flags:**

No snippet available

|                                                                                                                                                              |             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>D:\Work2\VSTS_Workspace\Onboarding\SourceCodes\Onboarding\Onboarding.WebAPI\Web.config, line 91 (Password Management: Password in Configuration File)</b> | <b>High</b> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|

#### Issue Details

**Kingdom:** Environment

**Scan Engine:** SCA (Configuration)

#### Sink Details

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.WebAPI/Web.config:91

**Taint Flags:**

No snippet available

## System Information Leak (2 issues)

### Abstract

Revealing system data or debugging information helps an adversary learn about the system and form a plan of attack.

### Explanation

An information leak occurs when system data or debugging information leaves the program through an output stream or logging function.

**Example:** The following code constructs a database connection string, uses it to create a new connection to the database, and prints it to the console.

```
string cs="database=northwind;server=mysqlServer...";
SqlConnection conn=new SqlConnection(cs);
...
Console.WriteLine(cs);
}
```

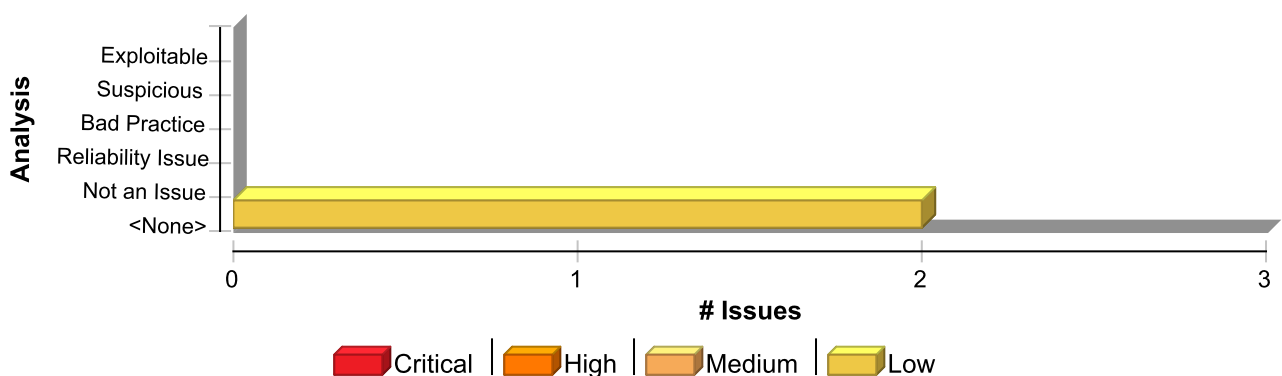
Depending on the system configuration, this information can be dumped to a console, written to a log file, or exposed to a remote user. In some cases the error message tells the attacker precisely what sort of an attack the system is vulnerable to. For example, a database error message can reveal that the application is vulnerable to a SQL injection attack. Other error messages can reveal more oblique clues about the system. In the example above, the search path could imply information about the type of operating system, the applications installed on the system, and the amount of care that the administrators have put into configuring the program.

### Recommendation

Write error messages with security in mind. In production environments, turn off detailed error information in favor of brief messages. Restrict the generation and storage of detailed output that can help administrators and programmers diagnose problems. Be careful, debugging traces can sometimes appear in non-obvious places (embedded in comments in the HTML for an error page, for example).

Even brief error messages that do not reveal stack traces or database dumps can potentially aid an attacker. For example, an "Access Denied" message can reveal that a file or user exists on the system.

### Issue Summary





## Engine Breakdown

|                         | SCA      | WebInspect | SecurityScope | Total    |
|-------------------------|----------|------------|---------------|----------|
| System Information Leak | 2        | 0          | 0             | 2        |
| <b>Total</b>            | <b>2</b> | <b>0</b>   | <b>0</b>      | <b>2</b> |

### System Information Leak

Low

Package: Onboarding.Common

D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/  
Onboarding.Common/Logger.cs, line 19 (System Information Leak)

Low

#### Issue Details

**Kingdom:** Encapsulation

**Scan Engine:** SCA (Data Flow)

#### Source Details

**Source:** Read ex

**From:** Onboarding.Common.Logger.LogError

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.Common/Logger.cs:19

```
16
17 public static void LogError(Exception ex)
18 {
19     _logger.Error(ex.Message, ex);
20 }
21
22 public static void LogError(string errMsg)
```

#### Sink Details

**Sink:** log4net.ILog.Error()

**Enclosing Method:** LogError()

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.Common/Logger.cs:19

**Taint Flags:** EXCEPTIONINFO, SYSTEMINFO

No snippet available

D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/  
Onboarding.Common/Logger.cs, line 19 (System Information Leak)

Low

#### Issue Details

**Kingdom:** Encapsulation

**Scan Engine:** SCA (Data Flow)

#### Source Details

**Source:** System.Exception.get\_Message()

**From:** Onboarding.Common.Logger.LogError

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.Common/Logger.cs:19

```
16
```

|                                                                                                                                 |            |
|---------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>System Information Leak</b>                                                                                                  | <b>Low</b> |
| <b>Package: Onboarding.Common</b>                                                                                               |            |
| <b>D:/Work2/VSTS_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.Common/Logger.cs, line 19 (System Information Leak)</b> | <b>Low</b> |

```

17 public static void LogError(Exception ex)
18 {
19     _logger.Error(ex.Message, ex);
20 }
21
22 public static void LogError(string errMsg)

```

### Sink Details

**Sink:** log4net.ILog.Error()

**Enclosing Method:** LogError()

**File:** D:/Work2/VSTS\_Workspace/Onboarding/SourceCodes/Onboarding/Onboarding.Common/Logger.cs:19

**Taint Flags:** EXCEPTIONINFO, SYSTEMINFO

No snippet available

